

MagicCode Super-Resolution Software

Specification

Document Version: v2.2.0. Release Date: September 30, 2026.

1. Document Overview

This specification defines capabilities, platforms, and functional boundaries of MagicCode Super-Resolution Software v2.2.0. It covers spatial and temporal GPU paths and spatial CPU paths.

1.4 Document Status

Version	Date	Status	Change
v2.0.0	September 2, 2026	Superseded	Temporal modes, session-API contract, platform matrix.
v2.1.0	September 4, 2026	Superseded	MC_Enable/MC_Disable public API (removed in v2.2.0).
v2.2.0	September 30, 2026	Current	nscaler public API (mc_nscaler_enable/control/disable/version); Linux x86_64 CPU spatial archive; Linux Vulkan temporal validated from source; spatial (1,2) band routing update.

2. System Requirements

Platform	GPU backends	Temporal
Android	Vulkan, OpenGLES	Temporal GPU yes; CPU temporal no
iOS / iPadOS / macOS	Metal	Temporal GPU yes; CPU temporal no
Windows	Vulkan, Direct3D 11, OpenGL 4.3	Temporal GPU yes; CPU temporal no
Linux x86_64	Vulkan (source build)	Shipped archive is CPU spatial (AVX2), no GPU objects; Linux Vulkan temporal validated from source (headless); CPU temporal no

- Input size [64, 4032]. Spatial scale [1, 8]. Temporal scale (1, 8).
- CPU spatial: x86_64 AVX2 and ARM64 NEON buffer paths. CPU temporal is not available.
- Linux x86_64: lib/linux/libmagic_sr.a ships the CPU spatial path (AVX2/FMA). Rebuild: tools/build_linux_lib.sh. Linux Vulkan temporal builds from source (MAGIC_ENABLE_VULKAN_TEMPORAL) and is validated headless; it is not part of the shipped archive.

3. Functional Capabilities

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

Spatial modes upscale a single frame. Temporal modes accumulate history and require depth, motion, jitter, and camera parameters through `mc_nscaler_input_frame_t` / `temporal_frame_t`.

Temporal Super-Resolution

Temporal super-resolution runs through the nscaler session API: `libmagic_sr.a` (Windows: `libmagic_sr.lib`) + `mc_interface.h` + `mc_nscaler_enable` / `mc_nscaler_control` / `mc_nscaler_disable`. v2.2.0 replaced the v2.0.0 `MC_Init` / `MC_Process` / `MC_Uninit` entry and removed the v2.1.0 `MC_Enable` helper (`mc_enable.h` is deleted). A NULL *handle on `mc_nscaler_enable` creates a session with defaults (2x `SPATIAL_SPEED`) and processes the first frame.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal `scaler_factor` is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- `TEMPORAL_SPEED_MODE` on Metal, Vulkan, and OpenGLS currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- `TEMPORAL_BALANCED_MODE` uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLS do not run edge-resolve. If `enable_sharpening` is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- `mc_nscaler_input_frame_t` / `mc_nscaler_output_frame_t`: caller-owned color and output GPU resources.
- `mc_nscaler_input_frame_t.frame`: non-NULL `temporal_frame_t*` with `struct_size = sizeof(temporal_frame_t)`.
- `temporal_frame_t.depth` and motion at input resolution.
- `jitter_offset_x/y` in input pixels, top-left, +X right, +Y down.
- `frame_index` / `reset_history`; `camera_near` / `camera_far` / `camera_fov_y` (radians).
- `gpu_context` at create (`mc_nscaler_control SET_PARAM`, or the first `mc_nscaler_enable`): Vulkan requires `physical_device + device`; D3D11 requires `device`; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan temporal requires `mc_nscaler_input_frame_t.command_buffer` to be a recording `VkCommandBuffer`. Metal `command_buffer` is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage `depth_reversed` / `depth_infinite` / `hdr_color` on `ctrl_param_t` (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- `motion_vector_scale_x/y` is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- `reactive` / `transparency` are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing `reactive` and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing `reactive` only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: `MAGICSR_TAAU_AUTO_MASK=0` and `MAGICSR_TAAU_HIST_REJECT=0` can turn those defaults off. They are not a customer configuration path.

4. Resource and synchronization contract

- Caller owns application GPU objects. The library owns internal history, scratch, and pipelines only.
- Vulkan: library does not begin/end/submit/wait. Restore of input layouts after compute; output remains GENERAL.
- Complete prior GPU work before `mc_nscaler_disable` or a size-changing enable.

5. Engine plugins

Unity and Unreal plugins expose Enable/Disable for spatial GPU textures. Full temporal integration is the Native session API described above. Plugin Enable is not a complete temporal entry.

6. Conformance notes

API support is not the same as device-run verification. Exact 2× has optimized temporal kernels; other rates in (1, 8] are supported by the API. Do not claim every scale or GPU vendor is verified.